Lutchman R. (231141979)

Role: Traffic Signals Management Module Developer

Introduction

The INT316D module, Internet Programming, has been one of the most valuable and challenging modules I have completed this semester. Before taking this course, I only had basic knowledge of web development and limited experience with Java programming. Throughout the semester, I learned important enterprise development concepts such as the evolution of Java Enterprise Edition (JEE), the Model-View-Controller (MVC) architecture, JavaServer Pages (JSP), servlets, exception handling, Enterprise JavaBeans (EJB), Java Persistence API (JPA), JPQL, and application security. These concepts were applied practically in our group project called SmartFlow, a traffic management system developed for the Witbank area. My role in the project was to design and implement the Traffic Signals Management Module, which allows traffic engineers to monitor and update traffic signal settings at different intersections. This reflection explains my contribution to the project and how I applied the knowledge gained from INT316D.

My Role in the SmartFlow Project

Traffic congestion and malfunctioning traffic lights are major problems in the Witbank area. Many intersections experience delays because signal settings are outdated or faults are not addressed quickly. Our SmartFlow system was designed to improve traffic management by creating a centralized platform where incidents, sensors, traffic signals, and users can be managed.

My responsibility was the Traffic Signals Management Module. I developed a page that displays all traffic signals in a table format. The table includes details such as the intersection name, current mode, cycle length, operational status, and last updated date. The modes available include Normal, Flashing, Manual Override, and Emergency Priority, while the status options include Operational, Faulty, Maintenance, and Testing.

I also implemented an edit modal that allows traffic engineers to update signal settings directly from the interface. Engineers can change the signal mode, cycle length, and operational status. When changes are submitted, the servlet updates the database and records the latest timestamp automatically. In addition, I added non-textbox controls such as checkboxes for signal features and color pickers for traffic priority colors. These controls demonstrated my understanding of different user interface components required in the assessment rubric.

Application of INT316D Topics

Evolution of JEE

Learning about the evolution of JEE helped me understand how enterprise applications developed over time. Early web applications relied heavily on servlets, which mixed Java code and HTML together, making applications difficult to maintain. JSP was introduced to improve the separation between presentation and business logic. Later, frameworks and patterns such as MVC became standard practice in enterprise development. Understanding this evolution helped me structure my module properly using servlets for control logic and JSP for presentation.

Model-View-Controller (MVC)

The MVC architecture played a major role in my module. The Model was represented by the Signal Java class, which stored information about traffic signals such as mode, cycle length, and status. The View was the signals.jsp page, which displayed signal information and forms for updates. The Controller was the SignalServlet, which handled requests from the user and communicated with the database.

Using MVC made the application easier to maintain because each component had a separate responsibility. The JSP focused only on displaying information, while the servlet handled processing and validation. This separation improved readability and reduced errors during development.

JSP and Servlets

JSP and servlets formed the foundation of my module. The JSP page used JSTL and Expression Language to display traffic signal data dynamically in a table. Each row included an Edit button that opened a modal containing the current signal information.

The SignalServlet handled both GET and POST requests. The doGet method retrieved signal information from the database and forwarded it to the JSP page. The doPost method processed updates submitted by traffic engineers. I learned that servlets are responsible for request handling and business logic, while JSP is mainly used for displaying information to users.

Exception Handling

Exception handling was an important part of my module because database operations can fail unexpectedly. I used try-catch-finally blocks to manage errors during database updates and queries. If an error occurred, the application displayed a user-friendly message instead of a system crash or stack trace.

I also used the finally block to ensure that database resources such as connections and statements were always closed. This prevented resource leaks and improved the reliability of the system. Through this process, I learned that professional applications must handle errors gracefully to provide a better user experience.

Enterprise JavaBeans (EJB)

Although our project did not fully implement EJB technology, I gained a strong understanding of its purpose. EJB simplifies enterprise application development by handling transactions, security, and scalability automatically.

For my module, I understood that a Stateless Session Bean could be used in a future version of the system. A method such as updateSignal() could automatically manage database transactions. If the update failed, the transaction would roll back automatically, ensuring that the database remained consistent. Learning EJB concepts helped me understand how large enterprise systems manage complex operations efficiently.

Java Persistence API (JPA) and JPQL

I learned that JPA provides a simpler way to manage database operations using object-relational mapping. Instead of writing SQL manually, classes can be mapped directly to database tables using annotations.

For example, the Signal class could be annotated with @Entity and @Table, while JPQL queries could retrieve objects directly from the database. This approach reduces boilerplate code and

makes applications more portable between databases. Although our project used JDBC for simplicity, understanding JPA and JPQL gave me insight into more advanced enterprise development practices.

Application Security

Security was important because only authorized engineers and administrators should access the traffic signals module. I implemented session checking to ensure that users were logged in before accessing the page. I also used a canAccessModule() function to verify user roles and permissions.

Prepared statements were used for database queries to prevent SQL injection attacks. I also learned the importance of additional security measures such as CSRF protection, password hashing, and HTTPS communication in real-world systems.

Challenges and Solutions

One of the biggest challenges I faced was implementing the edit modal correctly. Each traffic signal needed its own modal with pre-populated values. I solved this by dynamically generating unique modal IDs inside a JSP loop. This allowed the correct signal information to appear when the Edit button was clicked.

Another challenge involved updating the last_updated timestamp whenever a signal was changed. Initially, I attempted to generate the timestamp in Java, but later I used the MySQL NOW() function directly in the query because it ensured consistency with the database server.

I also found it challenging to implement the checkboxes and color pickers because I had limited JavaScript experience. After researching event listeners and testing different approaches, I successfully added interactive functionality to the controls.

What I Learned Throughout the Semester

The INT316D module taught me much more than programming syntax. I learned how enterprise applications are structured, how security is implemented, and why architecture patterns such as MVC are important. I also learned the importance of teamwork, communication, and testing during software development.

This project improved my problem-solving skills because many features required research and experimentation before they worked correctly. I also learned the value of testing edge cases and validating user input to make systems more reliable.

Conclusion

The SmartFlow project allowed me to apply the theoretical concepts learned in INT316D to a practical real-world system. My Traffic Signals Management Module demonstrated the use of MVC architecture, JSP, servlets, exception handling, security, and interactive user interface controls. Through this project, I gained valuable technical and teamwork experience that will help me in my future career as a software developer. I am proud of my contribution to the SmartFlow system and grateful for everything I learned throughout the semester.